Knoema Engine

An Open Runtime for Persistent NPCs, Synthetic Replay Research, and Reproducible Agent Simulation

Celovin | hello@celovin.com | Phase 30 draft | April 2026

Abstract

Knoema Engine is an open, MIT-licensed runtime for persistent social agents whose state is inspectable outside of a model prompt. The system combines persona definitions, short-term and long-term memory, directed relationship state, environment context, PAD-style emotion, event scheduling, deterministic logs, and optional LLM-backed decisions.

This revision extends that runtime with persona opt-in theory-of-mind tracking so belief state remains inspectable without forcing the feature on every agent.

This Phase 30 report updates the original MVP technical note with the 50-agent village experiment, formal benchmark bundle, scenario DSL, game SDK facades, a deterministic Sally-Anne harness, documentation infrastructure, and stricter public-safety boundaries.

1. Introduction

LLM-based agents can generate plausible language, but prompt-only designs often hide the state that a simulation, game, or research workflow needs to inspect. Persistent NPCs need memory and relationship state that a game can test. Research simulations need fixed configuration, deterministic replay, and exportable logs. Synthetic replay workflows need explicit safety boundaries around data use and claims.

Knoema addresses this engineering layer. It is not a general task-automation framework and it is not a hosted model service. It is a small runtime for building agents whose state is represented by ordinary package objects: persona, memory, relationships, environment, emotion, events, decisions, and logs.

Track	Primary surface	Phase 30 status
Games	Python, TypeScript, and GDScript NPC SDKs	Implemented
Synthetic replay	Scenario DSL and safety validator	Implemented
Academic simulation	Reproducible configs, logs, reports, and docs	Implemented

2. Related Work

Generative Agents showed that natural-language memory, planning, and reflection can produce believable behavior in interactive environments. Concordia frames generative agent-based modeling as grounded interaction among entities. The Sally-Anne task remains a compact probe for explicit false-belief reasoning. AutoGen, CAMEL, MetaGPT, ChatDev, Voyager, Reflexion, ReAct, and Toolformer explore ways to coordinate LLM behavior through roles, tools, self-reflection, or conversational protocols.

Classical agent-based modeling systems such as Mesa, NetLogo, MASON, Repast, Swarm, GAMA, and AnyLogic represent a long tradition of explicit model state, scheduling, and reproducible simulation loops. Knoema borrows this inspectable-loop discipline while adding natural-language memory and model-backed decision generation.

Retrieval-augmented generation, dense passage retrieval, approximate nearest-neighbor indexes, and vector stores provide the substrate for long-term agent context. Knoema currently uses deterministic hash embeddings in tests and examples to keep local runs reproducible.

3. Architecture

Knoema separates persistent state, decision generation, and adapters. State modules build an agent-specific context, the decision layer produces an action, and the simulator commits that action to logs, memories, relationship state, and downstream artifacts.

Module	Responsibility	MVP status
Persona	Identity, values, goals, prompt rendering	Implemented
Memory	Short-term FIFO plus SQLite and FAISS retrieval	Implemented
Relationship	Directed trust, familiarity, and interaction weight	Implemented
Environment	Time, location, conditions, and recent events	Implemented
Emotion	PAD state for valence, arousal, and dominance	Implemented
Theory of mind	Persona opt-in symbolic belief tracking	Implemented
Decision	Message rendering and strict action parsing	Implemented
Adapters	Dashboard, Godot, Unity, and SDK surfaces	Implemented

At every tick the simulator gathers persona, memory, relationship, environment, emotion, and optional theory-of-mind context for an agent. The decision engine renders messages, calls an LLMClient, and parses a strict action structure when possible. Deterministic local clients are used throughout tests and public demos.

The Scenario DSL expresses agents, environment, events, duration, metrics, and ethics flags in YAML. The parser returns typed scenario objects, the validator rejects unsafe or unsupported public-safety claims, and the serializer supports round trips for reproducibility.

4. Experiments and Evidence

Knoema's Phase 30 evidence is engineering-oriented. The experiments answer whether the runtime can generate reproducible artifacts, whether the memory policy improves deterministic proxy metrics over a naive full-context baseline, and whether a 50-agent scenario can be committed and replayed as a public artifact.

The 50-agent village experiment runs a deterministic week with 50 synthetic agents, 42 ticks, and 2,100 committed actions. Each agent contributes 42 actions. The run records 100 relationship edges, a deterministic seed of 20260418, a bit-for-bit artifact check, and zero provider cost because it uses local deterministic decisions.

50-agent metric	Value	Artifact
Agents	50	config.yaml
Ticks	42	metrics.json
Actions	2,100	sim_log.jsonl
Latency	p50 46.95 ms, p95 53.80 ms, p99 56.20 ms	metrics.json
Relationship edges	100	metrics.json
Provider cost	0.0 USD	deterministic-local

The formal benchmark bundle runs four scenarios across two approaches and three local model profiles, producing 24 deterministic runs. The comparison baseline is a naive full-context prompt policy.

Scenario	Knoema recall	Naive recall
A memory recall	0.789	0.500
B relationship dynamics	0.786	0.495
C narrative branching	0.784	0.492
D scalability	0.780	0.487

Across the deterministic matrix, the Knoema memory policy improves top-k memory recall proxies and token efficiency proxies in every scenario. The paired sign-test over the composite score reports p=0.000244. This is deterministic evidence over the benchmark proxy, not a real-world behavioral claim.

Phase 43 adds a persona opt-in theory-of-mind module and a deterministic Sally-Anne harness. The harness covers 20 structured cases and reproduces the expected search location in 20 out of 20 cases for an accuracy of 1.000 against a target gate of 0.800.

Theory-of-mind metric	Knoema	Reference status
Opt-in surface	Persona opt-in symbolic belief tracker	Concordia and Stanford are reference only
Sally-Anne reproduction	20/20 (1.000)	No public external score asserted

5. Applications

The game path treats the LLM as an optional dialogue provider behind a deterministic NPC contract. The current SDK response contains text, emotion, branch flags, and raw debugging state. This contract is intentionally small so a game can test behavior before connecting provider-backed dialogue.

The replay path uses fictional, non-identifying scenarios to inspect event traces. A scenario can define a known event sequence, run synthetic agents, and compare generated actions against process milestones. This is a replay and education workflow, not a prediction workflow.

The academic path emphasizes reproducible artifacts. Configurations, seeds, logs, metrics, dashboards, and reports remain close to the code. The MkDocs site and LaTeX report make the public API and evidence bundle easier to audit.

Application	Public artifact	Primary risk control
Game NPCs	SDK facades and adapters	Deterministic contract before live model calls
Synthetic replay	Scenario DSL and dashboard	No real personal data or prediction claims
Academic workflows	Reports and reproducibility tests	Config, seed, log, and hash checks

6. Discussion

The current evidence supports an engineering claim: explicit state, deterministic logs, and adapter contracts make persistent-agent systems easier to inspect and test. The repository demonstrates this through 136 local tests, benchmark artifacts, a public Playground, dashboards, SDK facades, and generated documentation.

The current evidence does not establish real-world behavioral validity, general human simulation accuracy, production game quality, or safe deployment in operational public-safety environments. The public-safety track is limited to synthetic replay and prevention-oriented analysis.

Threats to validity include deterministic local clients, lightweight hash embeddings, a simple naive baseline, and the absence of human evaluation. Larger-scale experiments need memory-store stress tests, model comparisons, and independent review.

Knoema uses repository-level checks to prevent private planning documents and unrelated entity references from entering public files. Runtime artifacts such as logs, SQLite databases, FAISS indexes, checkpoints, and documentation build output are ignored by git.

7. Safety Boundary

Public-safety examples are fictional, synthetic, and non-identifying. The system is not designed for prediction, suspect scoring, surveillance, or enforcement automation. Sensitive scenarios must include purpose notes, non-use boundaries, and reproducible artifacts.

Allowed	Rejected	Required
Fictional replay	Real personal data	Synthetic data notes
Education demos	Prediction or suspect ranking	Explicit non-use boundary
Reproducibility checks	Surveillance workflows	Config and log artifacts

8. Conclusion

Knoema Engine demonstrates a compact open runtime for persistent social agents. The Phase 30 version expands the original technical report with scenario DSL artifacts, SDK surfaces, a formal benchmark bundle, a 50-agent deterministic experiment, documentation infrastructure, and a stricter safety boundary.

The next research step is to replace deterministic proxy evaluation with controlled provider-backed runs, stronger semantic retrieval, human review of plausibility and safety, and fairer external baseline reproduction.

Appendix A. Artifact Map

The public repository is organized so each claim in the report points to a runnable artifact. Core runtime code lives in src/knoema. Scenario definitions and DSL docs live in examples/scenarios, schemas, and docs/dsl. Game integrations live in adapters and sdk. Benchmarks and committed results live in experiments and benchmarks. Documentation surfaces live in docs, website, and mkdocs.yml.

Artifact	Path	Purpose
Core runtime	src/knoema	State, decisions, events, memory, and simulation
Scenario DSL	src/knoema/dsl and schemas	Validated YAML scenario definitions
50-agent run	experiments/50_agent_village	Scale and reproducibility artifact
Formal benchmark	benchmarks/formal_report	Deterministic memory-policy comparison
Game SDKs	sdk and adapters	NPC response contracts and engine scaffolds
Docs site	mkdocs.yml and docs	API and workflow documentation

The LaTeX source includes figure and table fragments under paper/figures and paper/tables. The generated PDF preview is committed as paper/knoema_technical_report.pdf for reviewers who do not have a local TeX installation.

Appendix B. Reproducibility Checklist

Knoema treats reproducibility as an implementation requirement. Fixed seeds are part of scenario and experiment configuration. JSONL logs are committed for formal experiments when appropriate. Dashboard inspection consumes exported logs rather than simulator internals. Public-safety examples are fictional, synthetic, and non-identifying.

Check	Current status	Verification path
Same-seed determinism	Implemented	tests/reproducibility/test_deterministic_runs.py
Seed propagation	Implemented	tests/reproducibility/test_seed_propagation.py
Config serialization	Implemented	tests/reproducibility/test_config_serialization.py
JSONL replay	Implemented	tests/reproducibility/test_logs_replay.py
Scenario guardrails	Implemented	tests/test_phase22_dsl.py
Entity separation scans	Manual release gate	rg public-surface scans

Provider keys are not committed or persisted in public demos. The public Playground supports replay-only mode without a key and accepts optional user-supplied keys only for the current session.

References

Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442. https://arxiv.org/abs/2304.03442

Vezhnevets, A. S. et al. (2023). Generative Agent-Based Modeling with Actions Grounded in Physical, Social, or Digital Space Using Concordia. arXiv:2312.03664. https://arxiv.org/abs/2312.03664

Wimmer, H. and Perner, J. (1983). Beliefs about Beliefs: Representation and Constraining Function of Wrong Beliefs in Young Children's Understanding of Deception. Cognition, 13(1), 103-128.

Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. https://arxiv.org/abs/2308.08155

ter Hoeven, E. et al. (2025). Mesa 3: Agent-Based Modeling with Python in 2025. Journal of Open Source Software, 10(107), 7668. https://doi.org/10.21105/joss.07668

Johnson, J., Douze, M., and Jegou, H. (2017). Billion-Scale Similarity Search with GPUs. arXiv:1702.08734. https://arxiv.org/abs/1702.08734

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Sandve, G. K. et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLoS Computational Biology.

Wilkinson, M. D. et al. (2016). The FAIR Guiding Principles for Scientific Data Management and Stewardship. Scientific Data.

Mitchell, M. et al. (2019). Model Cards for Model Reporting. FAccT.

Gebru, T. et al. (2021). Datasheets for Datasets. Communications of the ACM.

Bommasani, R. et al. (2021). On the Opportunities and Risks of Foundation Models. arXiv:2108.07258.

Bender, E. M. et al. (2021). On the Dangers of Stochastic Parrots. FAccT.

Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.

Ouyang, L. et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS.

Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.

Reimers, N. and Gurevych, I. (2019). Sentence-BERT. EMNLP-IJCNLP.

Bonabeau, E. (2002). Agent-Based Modeling. PNAS.

Epstein, J. M. and Axtell, R. (1996). Growing Artificial Societies. Brookings Institution Press.

Railsback, S. F. et al. (2006). Agent-based Simulation Platforms. Simulation.

Liu, X. et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.

Liang, P. et al. (2022). Holistic Evaluation of Language Models. arXiv:2211.09110.

Gao, Y. et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

Sumers, T. R. et al. (2023). Cognitive Architectures for Language Agents. arXiv:2309.02427.